Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices basiques

Le but de cette séance est de découvrir et de mettre en pratique un paradigme de programmation largement utilisé : la programmation orientée objet (POO). Ce paradigme repose sur la création d'objets qui regroupent des données et des comportements, et qui peuvent interagir entre eux. Au fil des exercices, vous apprendrez à manipuler des objets et à maîtriser les notions fondamentales de classe, d'attribut, de méthode et d'encapsulation. À l'issue de cette séance, vous serez capables de concevoir des programmes mieux structurés et plus faciles à maintenir.

Les exercices de cette séance seront réalisés en Java et en Python.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Itération en Java

Question 1: (🕓 *5 minutes*)

Conseil

La méthode `String.valueOf` permet de convertir un nombre en chaîne de caractères. À l'inverse, la méthode `Integer.parseInt` permet de convertir une chaîne de caractères représentant un entier en une valeur de type `int`.

Qu'affiche le programme Java suivant (par souci de concision, nous omettons la déclaration de la classe et de la méthode `main`) :

```
1 String result = "";
2 String x = "5";
3 int j = 0;
4 for (j=Integer.parseInt(x); j !=0; j = j/2){
5     result = String.valueOf(j%2) + result;
6 }
7 System.out.println(result + j);
```

Choisissez parmi les éléments suivants :

(A) `5210`

(B) `1110`

(C) `1010`

(D) `1011`

(E) `java: bad operand types for binary operator '+'..`

Solution

(C). La boucle for calcule la représentation binaire de l'entier 5 stocké dans la chaîne `x`; cette représentation est `101`. L'affichage concatène ensuite la variable `result` et la valeur finale de `j`. Ici, `j` est définie hors de la portée de la boucle for, et cela est la raison pour laquelle on peut y accéder après la boucle for. À la sortie de la boucle for, la valeur de `j` est zéro. Le programme affiche donc `1010`.

2 Récursion en Java et Python

Question 2: (🕓 *10 minutes*) [Java et Python] **isPalindrome**

Conseil

En Java, pour obtenir le caractère à l'indice i d'une chaîne de caractères, `myString`, utilisez `myString.charAt(i)`. En Python, utilisez `myString[i]`.

Une sous-chaîne est obtenue en Java avec `myString.substring(i, j)` où i est l'indice du premier caractère et $j-1$ est l'indice du dernier caractère. En Python, utilisez `myString[i:j]`.

Un palindrome est une chaîne de caractères qui se lit de la même manière à l'envers. Plus précisément, soit f une fonction à valeurs booléennes qui renvoie vrai si `a` (de longueur n caractères) est un palindrome :

$$f(a) = f(a[1 : n-1]) \wedge (a[0] == a[n-1]) \tag{1}$$

Ici, $a[1 : n-1]$ est la sous-chaîne de a qui comprend tous les caractères à l'exception du premier caractère (le premier caractère étant à l'indice 0) et du dernier caractère (le dernier caractère étant à l'indice $n-1$).

Conseil

Revoir la question 16 du TP3 pour voir les règles concernant la notation des bornes d'une sous-chaîne de caractères.

On dit donc que a est un palindrome si le premier caractère et le dernier caractère sont égaux **et** la sous-chaîne entre les deux est un palindrome.

Le cas de base (une chaîne a d'un seul caractère) est :

$$f(a) = \text{vrai si } n = 1$$

et le cas de base d'une chaîne vide : $f(\texttt{""}) = \text{vrai}$.

Écrivez la méthode statique Java **`isPalindrome(String text)`** et la fonction Python **`isPalindrome(text)`** qui renvoient la valeur booléenne vraie si `text` est un palindrome.

Solution

[Java]

```java
public class Palindrome {
    public static void main(String[] args) {
        String pal1 = "abba";
        String pal2 = "ata";
        String nonPal1 = "abca";
        System.out.println(isPalindrome(pal1));
        System.out.println(isPalindrome(pal2));
        System.out.println(isPalindrome(nonPal1));
    }

    public static boolean isPalindrome(String text) {
        int length = text.length();
        if (length == 1 || length == 0) {
            return true;
        }
        return text.charAt(0) == text.charAt(length - 1)
                && isPalindrome(text.substring(1, length - 1));
    }
}
```

Solution

[Python]

```
1  def isPalindrome(text):
2      if len(text) == 0 or len(text) == 1:
3          return True
4      return text[0] == text[len(text) - 1] and
       isPalindrome(text[1:len(text) - 1])
5
6  print(isPalindrome("abba"))
7  print(isPalindrome("ata"))
8  print(isPalindrome("abca"))
```

3 Création de votre première classe en Java

Le but de cette première partie est de créer votre propre classe en Java. Cette classe, nommée `Dog`, représentera un chien. Elle aura plusieurs attributs et méthodes que vous implémenterez au fur et à mesure.

Question 3: (*10 minutes*) **Création de classe et encapsulation**
Commencez par créer une nouvelle classe `Dog` dans votre projet. Ensuite, créez les attributs suivants :

1. Un attribut `public` nommé `name` de type chaîne de caractères
2. Un attribut `private` nommé `tricks` de type liste de chaînes de caractères
3. Un attribut `private` nommé `race` de type chaîne de caractères
4. Un attribut `private` nommé `age` de type entier
5. Un attribut `private` nommé `mood` initialisé à 5 (correspondant à l'humeur du chien) de type entier
6. Un attribut de classe (`static`) `private` nommé `nbChiens` de type entier qui stocke le nombre d'objets `Dog` créés

Déclarez le constructeur public de la classe `Dog`. En Java, un constructeur porte le même nom que la classe et ne possède aucun type de retour. Il sert à initialiser les nouvelles instances en affectant aux attributs les valeurs des paramètres suivants :

1. Une chaîne de caractères `name`,
2. Une liste `tricks`,
3. Une chaîne de caractères `race`,
4. Un entier `age`.

Pour finir, le constructeur doit incrémenter l'attribut de classe `nbChiens`, qui garde en mémoire le nombre d'instances créées.

Conseil

Pour cet exercice, n'oubliez pas de préciser si vos attributs sont `public` ou `private`.
Le mot `static` correspond à un élément de classe (attribut ou méthode), cet élément pourra ensuite être appelé via la classe directement.
Pour attribuer des valeurs à vos attributs d'instance, utilisez le mot-clé `this.attribut`.
Pour accéder aux attributs de classe, utilisez `nomClasse.nomAttribut`.

Solution

```java
import java.util.LinkedList;
import java.util.List;

public class Dog {
    public String name; // Il est préférable de déclarer les attributs comme privés, mais celui-ci est
                        // public pour les buts de l'exercice.
    private List<String> tricks;
    private String race;
    private int age;
    private int mood = 5;
    private static int nbChiens = 0;

    public Dog(String name, List<String> tricks, String race, int age) {
        this.name = name;
        this.race = race;
        this.tricks = tricks;
        this.age = age;
        nbChiens++;
    }
}
```

Question 4: (*10 minutes*) **Getters et setters**

Il faut maintenant créer des méthodes de type getter et setter afin d'interagir avec les attributs `private` des instances de la classe. Les `getters` renverront les attributs souhaités tandis que les setters les modifieront. Les setters sont utilisés pour modifier la valeur d'attributs privés et ne renvoient rien.

Conseil

Exemple de `getters` et de `setters` :

```java
public String getName() { // Exemple de getter
    return name;
}

public void setName(String name) { // Exemple de setter
    this.name = name;
}
```

Vous pouvez directement accéder à des attributs publics en utilisant `nomInstance.attribut` à l'intérieur ou à l'extérieur de la classe. Cependant, comme la plupart des attributs de `Dog` sont privés, utilisez les getters et les setters pour les consulter et les modifier depuis une autre classe.

Créez les méthodes avec les en-têtes suivantes :

- `public List<String> getTricks()`
- `public String getRace()`
- `public int getAge()`
- `public int getMood()`
- `public void setTricks(List<String> tricks)`
- `public void setRace(String race)`
- `public void setAge(int age)`

- `public void setMood(int mood)`

Créez également la méthode de classe `public static int getNbChiens()`, qui retourne le nombre de `Dog` instanciés.

Conseil

Intellij vous permet de générer automatiquement certaines méthodes telles que les getters et setters. Vous pouvez consulter le lien suivant pour plus d'informations : `https://www.jetbrains.com/help/idea/generating-code.html#generate-delegation-methods`.
Toutefois, pour cet exercice, nous vous encourageons à le faire manuellement.

Solution

```
1       public List<String> getTricks() {
2           return tricks;
3       }
4
5       public int getAge() {
6           return age;
7       }
8
9       public int getMood() {
10          return mood;
11      }
12
13      public String getRace() {
14          return race;
15      }
16
17      public static int getNbChiens() {
18          return nbChiens;
19      }
20
21      public void setTricks(List<String> tricks) {
22          this.tricks = tricks;
23      }
24
25      public void setAge(int age) {
26          this.age = age;
27      }
28
29      public void setMood(int mood) {
30          this.mood = mood;
31      }
32
33      public void setRace(String race) {
34          this.race = race;
35      }
```

Question 5: (*5 minutes*) **Manipulation d'attributs — Listes**
Créez une méthode publique nommée `addTrick(String trick)` qui prend en entrée une chaîne de caractères et l'ajoute à la liste `tricks`.

Conseil

Les listes créées avec `List.of(...)` ne sont pas modifiables. Pour ajouter un `trick`, créez une copie modifiable de `tricks`, par exemple une `LinkedList<String>` (ou une `ArrayList<String>`), ajoutez-y le nouveau `trick`, puis affectez cette copie à `this.tricks`.

Solution

```
1  public void addTrick(String trick) {
2      LinkedList<String> temp = new
       LinkedList<String>(this.tricks);
3      temp.add(trick);
4      this.tricks = temp;
5  }
```

Question 6: (*5 minutes*) **Manipulation d'attributs**
Créez deux méthodes publiques permettant de modifier l'attribut `mood` d'un objet `Dog`. La méthode `leash()` décrémentera `mood` de 1 et `eat()` l'incrémentera de 3.

Solution

```
1  public void eat() {
2      this.mood = this.mood + 3;
3  }
4
5  public void leash() {
6      this.mood--;
7  }
```

Question 7: (*5 minutes*) **Manipulation d'attributs d'une autre instance**
Créez une méthode nommée `getOldest(Dog other)` qui prend comme argument un élément de type `Dog`, puis retourne le nom et l'âge du chien le plus âgé au format suivant : "`nomChien` est le chien le plus âgé : il a `ageChien` ans". Vous pouvez supposer que les deux chiens ont des âges différents.

Conseil

L'élément `other` est un objet de type `Dog` : vous pouvez donc appeler les méthodes définies précédemment pour consulter ses attributs.

Solution

```
1  public String getOldest(Dog other) {
2      if (other.getAge() < this.getAge()){
3          return this.name + " est le chien le plus âgé : il a "
       + this.age + " ans";
4      }
5      else{
6          return other.name + " est le chien le plus âgé : il a "
       + other.getAge() + " ans";
7      }
8  }
```

Question 8: (*5 minutes*) **Redéfinition de méthodes**
Redéfinissez la méthode `public String toString()` afin qu'elle retourne une chaîne de caractères contenant toutes les informations d'une instance de `Dog`. Ainsi, dans votre `main`, en affichant sur la console une instance de `Dog`, vous obtiendrez un texte au format suivant : "`nomChien` a `ageChien` ans, est un `raceChien` et a une humeur de `moodChien`. Il sait faire les tours suivants : `tricksChien`".

Conseil

La méthode `toString()` permet d'obtenir une représentation textuelle d'un objet. Elle est définie par défaut pour tout objet. Pour la redéfinir, on utilise le mot-clé `@Override` avant la méthode.

Solution

```
1  @Override
2  public String toString(){return this.name + " a " + this.age +
       " ans, est un " + this.race +
3              " et a une humeur de " + this.mood + ". Il sait
       faire les tours suivants : " + this.tricks;}
```

Pour contrôler que vos méthodes et attributs ont été implémentés correctement, exécutez le programme suivant :

```
1  import java.util.List;
2
3  public class Question8 {
4      public static void main(String[] args) {
5          Dog lola = new Dog("Lola", List.of("rollover"), "Bouvier", 10);
6          Dog tobi = new Dog("Tobi", List.of("rollover", "do a barrel"),
       "Doggo", 17);
7          System.out.println(lola.getAge());
8          System.out.println(lola.getMood());
9          System.out.println(lola.getRace());
10         System.out.println(lola.name);
11         System.out.println(lola.getTricks());
12         lola.setAge(13);
13         lola.setMood(8);
14         lola.setRace("Bouvier");
15         lola.name = "lola";
16         lola.setTricks(List.of("rollover", "do a barrel"));
17         lola.eat();
```

```
        lola.leash();
        lola.addTrick("sit");
        System.out.println(Dog.getNbChiens());
        System.out.println(lola.getOldest(tobi));
        System.out.println(lola);
    }
}
```

Vous devriez obtenir ce résultat :

```
10
5
Bouvier
Lola
[rollover]
2
Tobi est le chien le plus âgé : il a 17 ans
lola a 13 ans, est un Bouvier et a une humeur de 10. Il sait faire les
   tours suivants : [rollover, do a barrel, sit]
```

4 Notions de Programmation Orientée Objet en Python

Dans cette section, nous créerons pas à pas une classe `Point` contenant des attributs et des méthodes utiles. Dans votre IDE, créez un nouveau projet Python (Fichier > Nouveau > Projet). Dans un dossier de votre choix, créez un fichier `question9.py`.

Question 9: (*15 minutes*) **Classe `Point`**

- Créez une classe `Point` et définissez un constructeur prenant deux paramètres, `x` et `y`.

> **Conseil**
>
> Pour rappel, un constructeur est une fonction `__init__(self, ...)` que vous redéfinirez dans votre classe.

- Définissez deux attributs privés pour votre classe `Point`. Ces attributs seront les coordonnées x et y de vos points. Par défaut, assignez-leur les valeurs données dans le constructeur.

> **Conseil**
>
> À l'intérieur d'une classe, utilisez le mot-clé `self` pour accéder aux méthodes et attributs de l'instance que vous manipulez. En Python, pour spécifier qu'un attribut est privé, ajoutez deux traits de soulignement au nom de l'attribut (exemple : `__score=0`).

- Définissez les getters et les setters.

> **Conseil**
>
> En Python, le mot-clé `self` est l'équivalent de `this` utilisé en Java.

- Définissez une méthode `distance` qui prend en entrée l'instance du `Point` (`self`) et un autre `Point p2`. Cette méthode `distance` retournera la distance euclidienne entre le point `self` et `p2`.

> **Conseil**
>
> Pour rappel, la distance euclidienne entre deux points est définie par la formule $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Utilisez la fonction `sqrt()` de la bibliothèque `math` pour calculer la racine carrée. Pensez à importer la bibliothèque `math`.

- Définissez une méthode `milieu` qui prendra en entrée `self` et `p2` et qui retournera un objet `Point` situé exactement à mi-chemin entre ces deux points.

Conseil

Pour trouver les coordonnées d'un point M(x_M, y_M) situé au milieu du segment défini par des points A(x_A, y_A) et B(x_B, y_B), utilisez les formules suivantes : $x_M = \frac{x_A+x_B}{2}$ et $y_M = \frac{y_A+y_B}{2}$

- Redéfinissez la méthode `__str__()` dans la classe `Point` afin qu'elle retourne ses coordonnées. Par exemple, pour un point de coordonnées $(3, 2)$, la chaîne retournée sera : *Les coordonnées du Point sont : x = 3 et y = 2*

Solution

```
1  import math
2
3  class Point:
4      def __init__(self, x, y):
5          self.__x = x
6          self.__y = y
7
8      def get_x(self):
9          return self.__x
10
11     def get_y(self):
12         return self.__y
13
14     def set_x(self, x):
15         self.__x = x
16
17     def set_y(self, y):
18         self.__y = y
19
20     def distance(self, p2):
21         return math.sqrt((self.__x - p2.get_x())**2 + (self.__y
       - p2.get_y())**2)
22
23     def milieu(self, p2):
24         x_M = (self.__x + p2.get_x()) /2
25         y_M = (self.__y + p2.get_y()) / 2
26         M = Point(x_M, y_M)
27         return M
28
29     def __str__(self):
30         return "Les coordonnées du Point sont: x = " +
       str(self.get_x()) + " et y = " + str(self.get_y())
31
32 if __name__ == '__main__':
33     p = Point(3, 2)
34     p2 = Point(5,4)
35     print(p2)
36     print(str(p.distance(p2)))
37     print(str(p.milieu(p2)))
```

5 Notions d'héritage en Java

Le but de cette partie est de mettre en pratique les notions liées à l'héritage. Nous allons créer une classe `Livre` qui représentera notre classe-mère. Nous allons également créer deux classes filles, `LivreAudio` et `LivreIllustre`. Les classes filles hériteront des attributs et méthodes de la classe-mère.

Question 10: (*20 minutes*) **Création des différentes classes**
Créez la classe-mère `Livre` avec les caractéristiques suivantes :

- un attribut `private` nommé `titre` de type chaîne de caractères,
- un attribut `private` nommé `auteur` de type chaîne de caractères,
- un attribut `private` nommé `annee` de type entier,
- un attribut `private` nommé `note` initialisé à `-1`,
- le constructeur `public Livre(String titre, String auteur, int annee)`,
- une méthode `public void setNote(int note)` qui permet de définir l'attribut `note`,
- une méthode `public int getNote()` qui permet de retourner l'attribut `note`,
- une méthode `public String toString()` qui retournera le titre, l'auteur, l'année et la note d'un ouvrage (redéfinir cette méthode permettra d'afficher un objet `Livre` en utilisant `System.out.println()`).

Attention, si la `note` n'a pas été modifiée et qu'elle vaut toujours `-1`, affichez `Note: pas encore attribuée` au lieu de `Note: {note}` via la méthode `toString()`.

Créez les classes filles avec les caractéristiques suivantes :
`class LivreAudio extends Livre`

- un attribut `private String` nommé `narrateur`,
- le constructeur `public LivreAudio(String titre, String auteur, int annee, String narrateur)`, qui initialise les attributs hérités et l'attribut `narrateur`.

`class LivreIllustre extends Livre`

- un attribut `private String` nommé `illustrateur`,
- le constructeur `public LivreIllustre(String titre, String auteur, int annee, String illustrateur)`, qui initialise les attributs hérités et l'attribut `illustrateur`.

Voici le squelette du programme à compléter :

```
1 class Livre {
2
3 }
4 class LivreAudio extends Livre {
5
6 }
7 class LivreIllustre extends Livre {
8
9 }
```

Conseil

- En Java, lors de la déclaration d'une classe, le mot-clé `extends` permet d'indiquer qu'il s'agit d'une classe fille de la classe indiquée. Pour initialiser la partie héritée d'un objet, appelez le constructeur de la classe-mère avec `super(attributMere1, attributMere2, attributMere3, etc.);`.
- Dans ces exercices, placez l'appel au constructeur de la classe mère `super(...)` au début du constructeur de la classe fille. Depuis Java 25, certaines instructions, comme la validation des paramètres, peuvent toutefois précéder cet appel.
- Vous pouvez vous servir de `'\n'` dans une chaîne de caractères pour effectuer un retour à la ligne lors de l'affichage d'une chaîne de caractères.

Solution

```
1  public class Livre {
2
3      private String titre;
4      private String auteur;
5      private int annee;
6      private int note = -1;
7
8      public Livre(String titre, String auteur, int annee){
9      this.titre = titre;
10     this.auteur = auteur;
11     this.annee = annee;
12 }
13
14 public int getNote(){
15     return this.note;
16 }
17
18 public void setNote(int note) {
19     this.note = note;
20 }
21
22 @Override
23 public String toString() {
24     if (note == -1){
25         return "A propos du livre\n------------------\nTitre :
       " +titre+ "\nAuteur : "+auteur+ "\nAnnée : "+annee+ "\nNote
       : pas encore attribuée";
26         }
27     else{
28         return "A propos du livre\n------------------\nTitre :
       " +titre+ "\nAuteur : "+auteur+ "\nAnnée : "+annee+ "\nNote
       : "+note;
29         }
30     }
31 }
```

Solution

```
1  class LivreAudio extends Livre {
2      private String narrateur;
3
4      public LivreAudio(String titre, String auteur, int annee,
       String narrateur){
5      super(titre, auteur, annee);
6      this.narrateur = narrateur;
7      }
8  }
```

Solution

```
1  class LivreIllustre extends Livre {
2
3      private String illustrateur;
4
5      public LivreIllustre(String titre, String auteur, int
       annee, String illustrateur) {
6      super(titre, auteur, annee);
7      this.illustrateur = illustrateur;
8      }
9  }
```

Question 11: (*10 minutes*) **Méthode et héritage**
Maintenant que vous avez créé la classe-mère et ses classes filles, procédez comme suit :

1. Dans la classe `LivreAudio`, redéfinissez la méthode `toString()` afin qu'elle retourne les informations définies dans la classe `Livre`, suivies de la valeur de l'attribut `narrateur`.
2. Faites de même dans la classe `LivreIllustre` afin d'ajouter la valeur de l'attribut `illustrateur`.
3. Dans la méthode `main`, déclarez une variable de type `Livre` et initialisez-la avec une instance de `LivreAudio`. Utilisez les valeurs suivantes : titre `Hamlet`, auteur `Shakespeare`, année `1609` et narrateur `William`.
4. Attribuez la note `5` à ce livre à l'aide de la méthode setter définie précédemment.
5. Déclarez une deuxième variable de type `Livre` et initialisez-la avec une instance de `LivreIllustre`. Utilisez les valeurs suivantes : titre `Les Misérables`, auteur `Victor Hugo`, année `1862` et illustrateur `Gustave Brion`. Ne lui attribuez aucune note afin de vérifier le comportement prévu pour la valeur `-1`.
6. Affichez les informations des deux livres.

Conseil

Réfléchissez à la différence entre le type d'une variable et le type de l'objet qu'elle référence. Pour réutiliser le comportement de la classe parente dans une méthode redéfinie, pensez au mot-clé `super`.

Solution

```
1  class LivreAudio extends Livre {
2
3      private String narrateur;
4
5      public LivreAudio(String titre, String auteur, int annee,
       String narrateur) {
6          super(titre, auteur, annee);
7          this.narrateur = narrateur;
8      }
9
10     @Override
11     public String toString() {
12         return super.toString() + "\nNarrateur: " + narrateur;
13     }
14 }
```

Solution

```
class LivreIllustre extends Livre {

    private String illustrateur;

    public LivreIllustre(String titre, String auteur, int
    annee, String illustrateur) {
        super(titre, auteur, annee);
        this.illustrateur = illustrateur;
    }

    @Override
    public String toString() {
        return super.toString() + "\nIllustrateur: " +
    illustrateur;
    }
}
```

Solution

```
public class Question11 {

    public static void main(String[] args) {
        Livre livreAudio = new LivreAudio("Hamlet",
    "Shakespeare", 1609, "William");
        livreAudio.setNote(5);
        Livre livreIllustre = new LivreIllustre("Les
    Misérables", "Victor Hugo", 1862, "Gustave Brion");

        System.out.println(livreAudio);
        System.out.println(livreIllustre);
    }
}
```

Lorsque toutes les étapes auront été effectuées, exécutez le programme suivant :

```
public class Question11 {

    public static void main(String[] args) {
        Livre livreAudio = new LivreAudio("Hamlet", "Shakespeare",
    1609, "William");
        livreAudio.setNote(5);
        Livre livreIllustre = new LivreIllustre("Les Misérables",
    "Victor Hugo", 1862, "Gustave Brion");

        System.out.println(livreAudio);
        System.out.println(livreIllustre);
    }
}
```

Vous devriez obtenir :

```
A propos du livre
------------------
Titre : Hamlet
```

```
Auteur : Shakespeare
Année : 1609
Note : 5
Narrateur: William
A propos du livre
------------------
Titre : Les Misérables
Auteur : Victor Hugo
Année : 1862
Note : pas encore attribuée
Illustrateur: Gustave Brion
```